Manual de Git y GitHub CLI desde terminal

Flujo de trabajo para el equipo de mecatrónica: descargar el repositorio, editar sketches en Arduino IDE y proponer cambios mediante Pull Request, todo desde la terminal.

1. Instalación

Sistema	Git	GitHub CLI (gh)
Windows (PowerShell)	`winget install --id Git.Git -e`	`winget install --id GitHub.cli -e`
macOS (Terminal)	`brew install git`	`brew install gh`
Linux (Debian/Ubuntu)	`sudo apt install git`	`sudo apt install gh`

Cierra y abre la terminal después de instalar. Verifica:

```
git --version
gh --version
```

2. Configuración inicial (una sola vez)

1. Identidad que aparecerá en tus commits (usa el correo de tu cuenta de GitHub):

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu_correo@ejemplo.com"
git config --global init.defaultBranch main
git config --global pull.rebase false
```

2. Iniciar sesión en GitHub:

```
gh auth login
```

Elige: `GitHub.com` → `HTTPS` → `Yes` (autenticar Git con tus credenciales de GitHub) → `Login with a web browser`. Copia el código de 8 caracteres, pégalo en el navegador y autoriza.

3. Comprobar la sesión:

```
gh auth status
```

3. Clonar el repositorio (una sola vez)

```
gh repo clone <organizacion-o-usuario>/<repositorio>
cd <repositorio>
```

Esto crea una carpeta local con todo el proyecto. Trabaja siempre dentro de esa carpeta.

4. Flujo de trabajo diario

Regla: nunca se trabaja directamente sobre `main`. Cada tarea va en su propia rama y se integra con un Pull Request (PR).

1. Actualizar tu copia local antes de empezar:

```
git checkout main
git pull
```

2. Crear una rama para tu tarea (nombre corto, sin espacios, con el tipo y el tema):

```
git checkout -b feature/sensor-temperatura
```

3. Abrir y editar el sketch en Arduino IDE. Guarda con Ctrl+S (Cmd+S en macOS). Cierra o deja abierto el IDE; Git solo ve los archivos guardados.
4. Revisar qué cambió:

```
git status
git diff
```

5. Preparar los archivos y crear el commit (un commit por cambio lógico, con mensaje claro):

```
git add sketch_sensor/sketch_sensor.ino
git commit -m "Agrega lectura de temperatura con DS18B20"
```

Para agregar todo lo modificado: `git add .` (revisa antes con `git status` que no entren archivos ajenos al proyecto).

6. Subir la rama a GitHub (la primera vez):

```
git push -u origin feature/sensor-temperatura
```

Las siguientes veces basta `git push`.

7. Crear el Pull Request:

```
gh pr create --base main --title "Lectura de temperatura DS18B20" --body
"Agrega lectura y filtrado. Probado en Arduino Mega."
```

8. Esperar la revisión. Si piden cambios, repite los pasos 3 a 6 en la misma rama; el PR se actualiza solo.
9. Después de que el PR se fusione, limpiar:

```
git checkout main
git pull
git branch -d feature/sensor-temperatura
```

5. Comandos útiles de gh

Acción	Comando
Ver mis PR abiertos	`gh pr list --author "@me"`
Ver estado de un PR	`gh pr view <número>`
Descargar la rama de un PR para probarlo	`gh pr checkout <número>`
Abrir el repo en el navegador	`gh repo view --web`
Abrir el PR actual en el navegador	`gh pr view --web`
Ver issues asignados	`gh issue list --assignee "@me"`

6. Reglas para proyectos Arduino

- La carpeta del sketch debe llamarse igual que el archivo `.ino` (`sketch_sensor/sketch_sensor.ino`). Arduino IDE lo exige.
- Nunca subir contraseñas WiFi, tokens ni claves API. Ponerlas en un archivo `secrets.h` local, agregarlo al `.gitignore` y dejar un `secrets.example.h` con valores falsos.
- No subir archivos de compilación ni configuraciones personales del IDE.
- Si instalas una librería nueva, anótala en el README del repo (nombre y versión) para que todos usen la misma.
- Indica en el PR qué placa y qué puerto se usó para probar.
- Un PR por tarea: cambios pequeños se revisan más rápido.

Contenido mínimo recomendado de `.gitignore` en la raíz del repo:

```
secrets.h
build/
*.hex
*.elf
*.bin
.vscode/
.DS_Store
```

7. Conflictos y errores comunes

Situación	Qué hacer
`git push` rechazado (rejected, non-fast-forward)	`git pull` en tu rama, resolver y volver a `git push`
Conflicto al hacer `git pull`	Abrir los archivos marcados con `<<<<<<<`, dejar la versión correcta, borrar las marcas, luego `git add <archivo>` y `git commit`
Me equivoqué en el último commit (aún sin push)	`git commit --amend -m "Mensaje corregido"`
Quiero descartar cambios de un archivo sin commit	`git restore <archivo>` (los cambios se pierden)
Hice cambios estando en `main` sin querer	`git checkout -b feature/mi-tarea` (los cambios pasan a la nueva rama)
Pide autenticación o falla el acceso	`gh auth status` y, si hace falta, `gh auth login`
No sé en qué rama estoy	`git branch` (la activa lleva `*`)

No usar `git push --force` ni `git reset --hard` sin consultar antes al responsable del repositorio.

8. Hoja rápida

```
git checkout main && git pull          # actualizar
git checkout -b feature/<tema>         # nueva rama
git status                             # ver cambios
git add <archivo>                      # preparar
git commit -m "mensaje claro"          # confirmar
git push -u origin feature/<tema>      # subir rama
gh pr create --base main               # abrir Pull Request
```